

**E-R Model:** E-R Model stands for Entity-Relationship Model, also known as E-R Diagram. It was proposed by Peter Chen in 1971 to create a uniform convention which can be used for relational database.

E-R model helps us to systematically analyze data requirements to produce a well-designed database.

The ER model is used to define the conceptual view of database. The work of the ER Model revolves around real-world entities and the associations between these objects. At view level, the ER model is considered a good option for designing databases.

### Why we use ER Diagrams?

- Helps to describe entities, attributes, relationships.
- Provide a preview of how all your tables should connect, what fields are going to be on each table.
- ER diagrams are translatable into relational tables which allow you to build databases quickly.
- ER diagrams can be used by database designers as a blueprint for implementing database.
- The database designer gains a better understanding about the database with the help of ER diagram.

### Advantages

- **Simple:** If we know the relationship between the attributes and the entities we can easily build the ER Diagram for the model.
- **Effective Communication Tool:** This model is used widely by the database designers for communicating their ideas.
- **Easy Conversion to any Model:** This model can be converted to any other model like relational, network, hierarchical model etc.

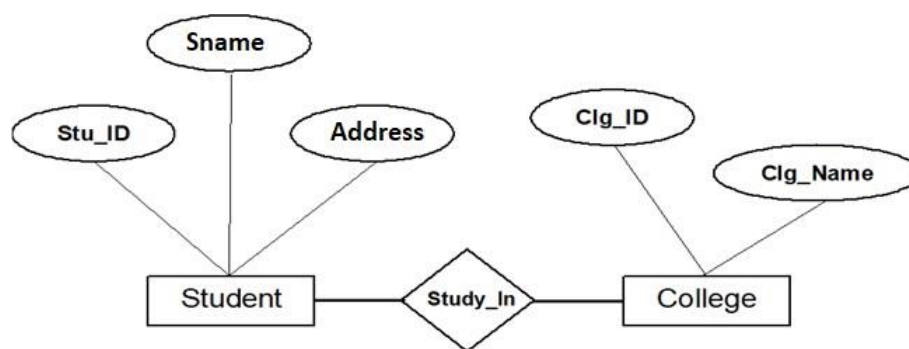
### Disadvantages

- **No industry standard for notation:** means one developer might use notations which are not understood by other developers.
- **Hidden information:** As it is a high-level view so there are chances that some details of information might be hidden.

**Components of the ER Diagram:** This model is based on three basic concepts:

1. Entities.
2. Attributes.
3. Relationships.

### Example of ER diagram:



E-R Diagram

**Entity and Entity Set:** An **entity** is a **real-world thing** which is distinguishable from other. An entity can be place, person, object, event or a concept. For example, in a company database, employees, dealers, orders, and products can be considered as entities. Every entity is made up of some '**attributes**' which represent that entity. It is represented by a **rectangle**.

**Examples:**

- **Person:** Employee, Student, Patient.
- **Place:** Store, Building.
- **Object:** Machine, product, and Car.
- **Event:** Sale, Registration, Renewal.
- **Concept:** Account, Course.

A collection of **similar types of entities** put together is known as an **entity set**. An entity set may contain entities with attribute sharing similar values. For example, an Employee set may contain all the employees of a company. Entity sets need not be disjoint.

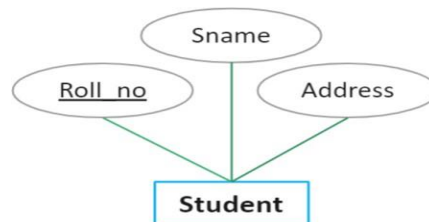


**Fig:** Representation of Entity.

**Types of Entities:** There are two types of entities. They are:

1. Strong Entity.
2. Weak Entity.

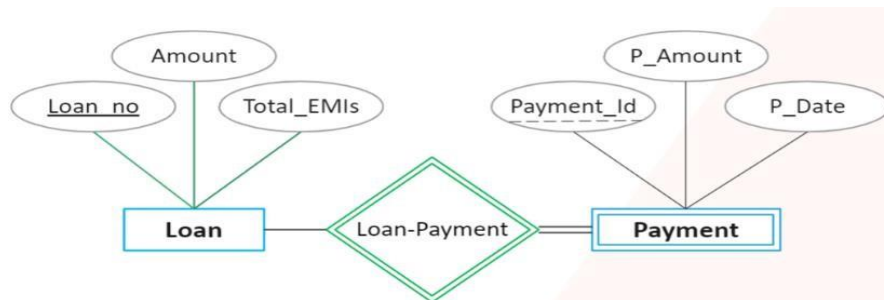
**Strong Entity:** Strong entities are those entity types which have a key attribute. The primary key helps in identifying each entity uniquely. It is represented by a rectangle. In the below example, roll\_no identifies each row of the table uniquely and hence, we can say that STUDENT is a strong entity.



**Fig:** Representation of Strong entity.

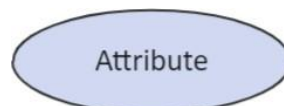
**Weak Entity:** Weak entity doesn't have a key attribute. Weak entity type can't be identified on its own. Its existence depends upon some other strong entity. For example, there can be children only if the parent exists. There can be no independent existence of children.

A weak entity is represented by a **double outlined rectangle**. The relationship between a weak entity type and strong entity type is called an **identifying relationship** and shown with a **double outlined diamond**. This representation can be seen in the diagram below. **Example:** If we have two entities of Loan (Loan\_no, amount, Total\_EMIs) and Payment (Payment\_Id, P\_Amount, and P\_Date). In this case, we cannot identify the **Payment** on its own, if there is no **Loan**.



**Fig:** Representation of Weak entity.

- **Attributes:** An attribute is a property or characteristic of the entity that holds it. For example, a Student might have attributes: roll\_no, name, address, phone\_no, etc. It is represented by an ellipse/oval.



**Fig:** Representation of Attribute.

There exists a specific **domain** or **set of values** for each attribute from where the attribute can take its values.

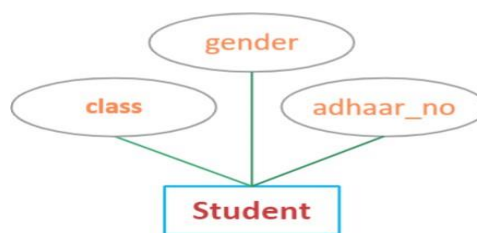
For example, a student's name cannot be a numeric value. It has to be alphabetic. A student's age cannot be negative, etc.

**Types of Attributes:** In ER diagram, attributes associated with an entity set may be of the following types-

1. Simple attribute.
2. Composite attribute.
3. Single valued attribute.
4. Multi valued attribute.
5. Derived attribute.
6. Key attributes.

**1. Simple attribute:** Simple attributes are atomic values, which cannot be divided further.

**Example:**

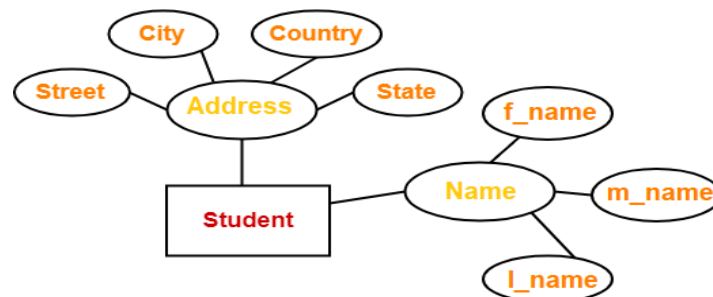


**Fig:** Representation of Simple attributes

Here, student's **class, gender, and DOB** are **atomic** values which cannot be divided further.

**2. Composite attribute:** A composite attribute is made up of more than one simple attribute and a composite attribute can be further divisible.

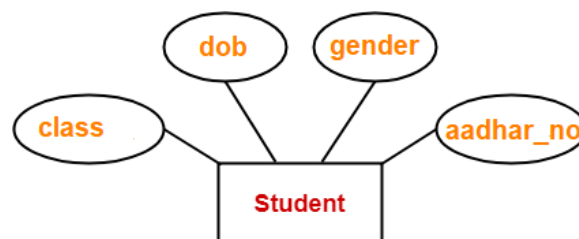
**Example:** Here, the attributes "Name" and "Address" are composite attributes as they are composed of many other simple attributes.



**Fig:** Representation of Composite attributes

**3. Single Valued Attribute:** These are the attributes which can take only one value at a time.

**Example:**

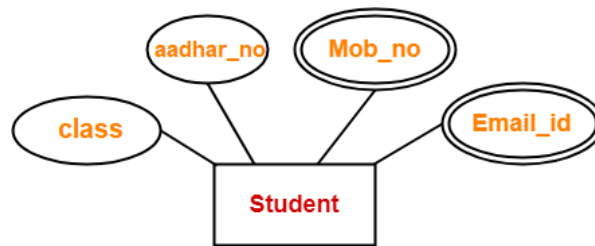


**Fig:** Representation of Single valued attributes

Here, all the attributes are single valued attributes as they can take only one at a time.

**4. Multi-Valued Attribute:** These are the attributes which can take more than one value at a time and they are represented by a double ellipse.

**Example:**

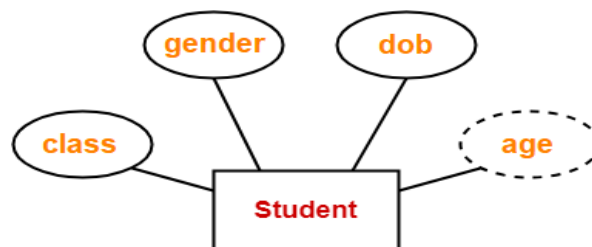


**Fig:** Representation of Multi - valued attributes

Here, the attributes “**Mob\_no**” and “**Email\_id**” are multi-valued attributes as they can take more than one value.

**5. Derived Attribute:** These are the attributes which can be derived from other attribute(s) and they are represented by dashed ellipses.

**Example:**

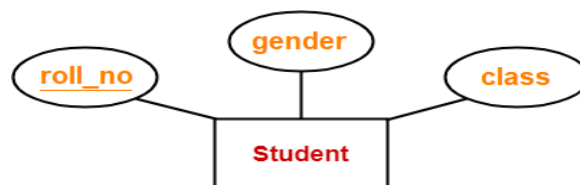


**Fig:** Representation of Derived attributes

Here, “**age**” is a derived attribute as it can be derived from the attribute “**dob**”.

**6. Key Attribute:** These are the attributes which can identify an entity uniquely in an entity set and key attributes should be underlined.

**Example:**



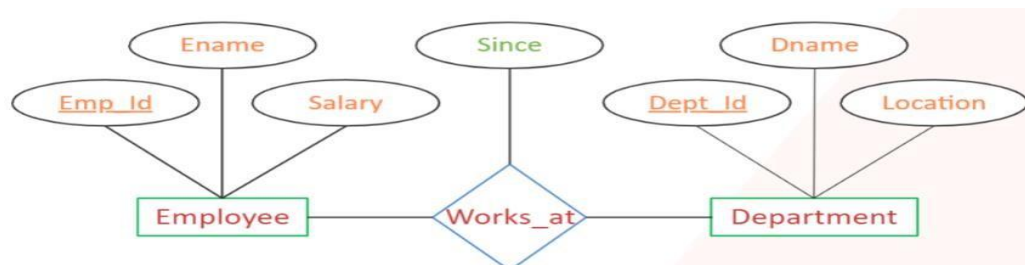
**Fig:** Representation of Key attributes.

Here, the attribute “**roll\_no**” is a key attribute as it can identify each student uniquely.

➤ **Relationship:** A relationship is an association among two or more entities. Entities take part in the relationship. It is represented by a diamond shape.

For example, an employee works at a specific department. Here, “**works at**” is a relationship and this is the relationship between an Employee entity and a Department entity.

Like entities, a relationship too can have attributes. These attributes are called **descriptive attributes**. Here “**Since**” is the descriptive attribute.



**Fig:** Example of **Works at** relationship

**Relationship Set:** A set of relationships of similar type is called a relationship set.

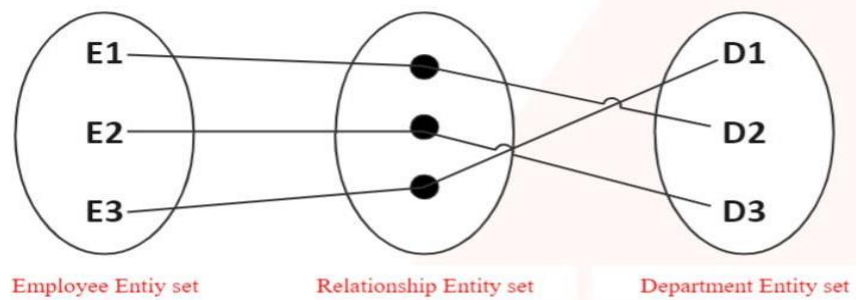


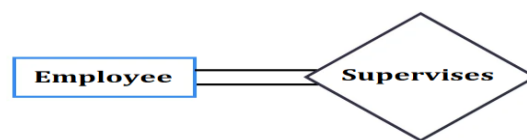
Fig. Relationshipset representation

**Degree of a Relationship:** The number of entity sets that participate in a relationship set is termed as the degree of that relationship. Degree of above relationship is 2.

**Types of Relationships:** On the basis of degree of a relationship set, a relationship can be classified into the following types:

1. Unary relationship.
2. Binary relationship.
3. Ternary relationship.
4. N-ary relationship.

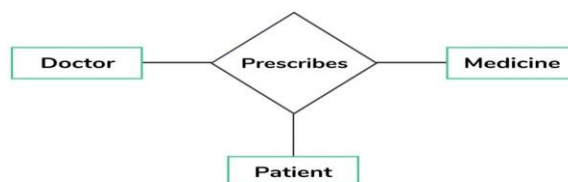
**1. Unary relationship:** When there is a relationship between two entities of the same type is known as Unary relationship. It is known as a recursive relationship. This means that the relationship is between different instances of the same entity type. For example, an employee can supervise group of employees. Hence, this is a recursive relationship of entity employee with itself.

Unary relationship  
(degree 1)

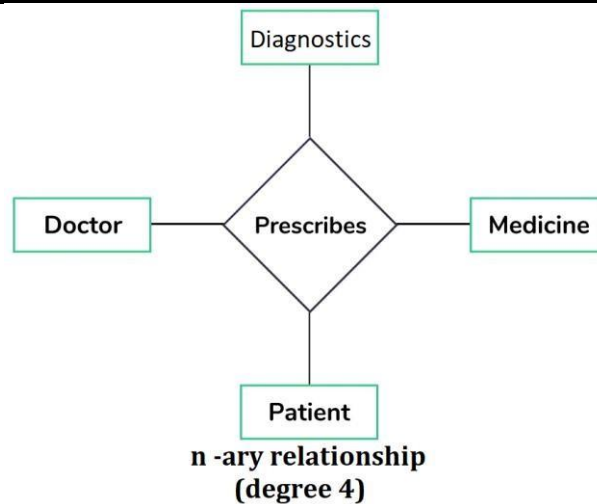
**2. Binary Relationship:** When there are exactly two entity sets participating in a relationship then such type of relationship is called binary relationship. For example, a student enrolls a course, here, “enrolls” is a relationship and this is the relationship between a Student entity and a Course entity.

Binary relationship  
(degree 2)

**3. Ternary relationship:** When there are exactly three entity sets participating in a relationship then such type of relationship is called ternary relationship and For example: In the real world, a patient goes to a doctor and doctor prescribes the medicine to the patient, three entities Doctor, Patient and Medicine are involved in the relationship “prescribes”.

Ternary relationship  
(degree 3)

**N-ary relationship:** When more than three entity sets are participating in a relationship, then such type of relationship is called an N-ary relationship. For example: In the real world, a patient goes to a doctor and doctor prescribes the diagnosis and medicine to the patient, four entities Doctor, Patient, Medicine and Diagnostics are involved in the relationship “prescribes”.



**Mapping Cardinalities / Constraints:** It defines how many entities in entity set A can be related to entities in entity set B in a relationship set R. (or) it defines number of instances participated in a relationship. It is most useful in describing the relationship sets that involve more than two entity sets. For binary relationship set R on an entity set A and B, there are four possible. These are as follows:

1. **One to one (1:1)**
2. **One to many (1:M)**
3. **Many to one (M: 1)**
4. **Many to many (M:M)**

**1. One to One (1:1):** An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A. Symbol used:

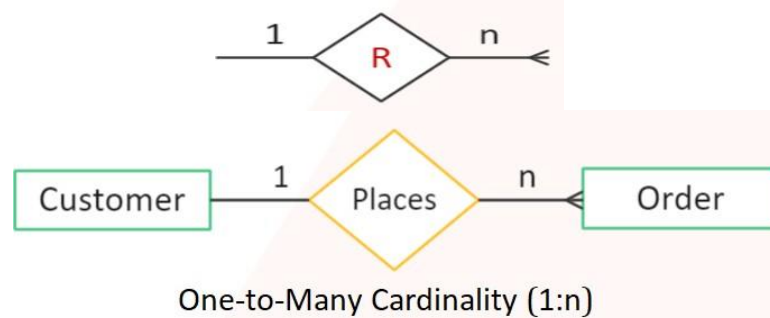


For example, assume that the **only male can be married to only one female and one female can be married to only one male**, this can be viewed as one-to-one cardinality.



**2. One to Many (1: M):** An entity in A is associated with any (zero or more) entities of B, and an entity in B can be associated with at most one entity in A. Symbol used:

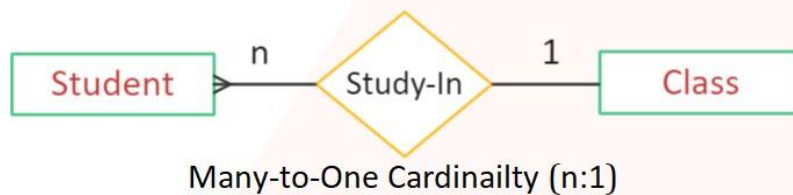
For example, a customer can place multiple orders from a website/shop.



**3. Many to One (M: 1):** An entity in A is associated with at most one entity in B and an entity in B can be associated with any (zero or more) number of entities in A. Symbol used:



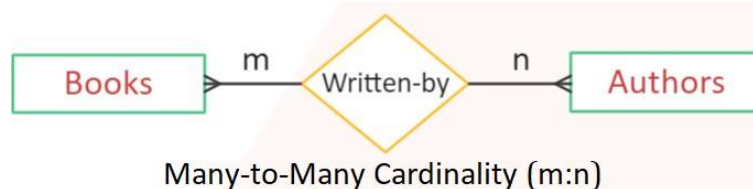
For example: A student can belong to at most one class but one class can have many students.



**4. Many to Many (M: N):** An entity in A is associated with any number of entities in B and an entity in B is associated with any number of entities in A. Symbol used:



For example: Here many books are written by many authors and many authors write many books.



#### Participation Constraints:

- There are two types of participation constraints — **total** and **partial**.
- The participation of an entity set **E** in a relationship set **R** is said to be **total** if **every entity in E participates in at least one relationship in R**.
- If only **some entities in E participate in relationships in R**, the participation of entity set E in relationship R is said to be **partial**.
- **Double line** indicates the **total participation** constraint.
- **Single line** indicates the **partial participation** constraint.
- For example, a student enrolled for course, if we are representing this as a diagram,



The participation of entity set **course** in **enrolled** relationship set is **partial** because a course may or may not have students enrolled in.

The participation of entity set **student** in **enrolled** relationship set is **total** because every student is expected to relate at least one course through the enrolled relationship set

**Cardinality Constraints** specify maximum participation i.e., **At most**.

**Participation Constraints** specify minimum participation: **At least** or Minimum.



**Extended Features of E – R Model:** As the complexity of data increased in the late 1980s, it became more and more difficult to use the traditional ER Model for database modeling. Hence some improvements or enhancements were made to the existing ER Model to make it able to handle the complex applications better.

Hence, as part of the **Enhanced ER Model**, three new concepts were added to the existing ER Model, they were:

1. Generalization.
2. Specialization.
3. Aggregation.

### Features of Extended E-R Model

- EER creates a design more accurate to database schemas.
- It reflects the data properties and constraints more precisely.
- It includes all modeling concepts of the ER model.
- Diagrammatic technique helps for displaying the EER schema.
- It includes the concept of specialization and generalization.

### Enhanced ER modeling

- ✓ Enhanced features of ER model are specialization and Generalization.
- ✓ **Specialization** - Specialization is the process of identifying subsets of an entity set (the **superclass**) that share some distinguishing characteristic.
- ✓ **Generalization** - Generalization consists of identifying some common characteristics of a collection of entity sets and creating a new entity set that contains entities possessing these common characteristics.
- ✓ We can inherit the properties of super class into sub class by ISA relationship.

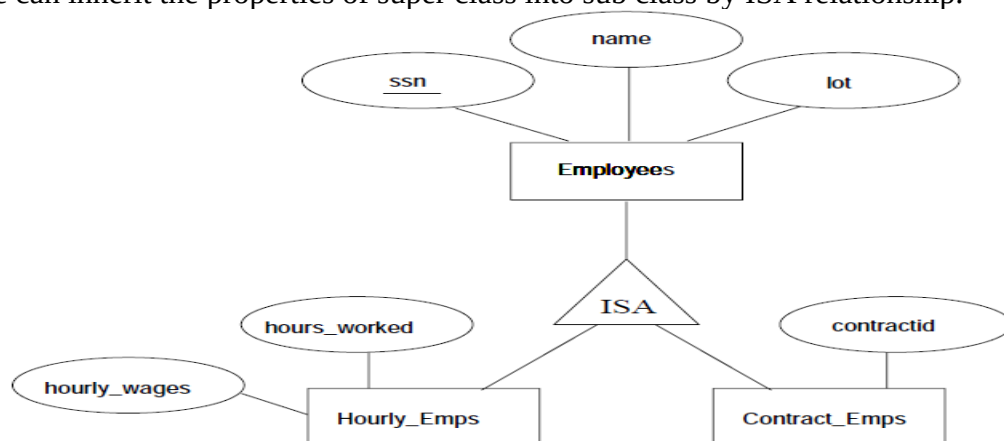


Figure: An example of Specialization and Generalization.

- ✓ We can specify two kinds of constraints with respect to ISA hierarchies, namely, *overlap* and *covering* constraints. Overlap constraints determine whether two subclasses are allowed to contain the same entity. Covering constraints determine whether the entities in the subclasses collectively include all entities in the superclass.

**Notations:** E-R Diagram is a visual representation of data that describes how data is related to each other using different symbols and notations. Following are the major components and its symbols in E- R Diagrams:

- **Rectangle:** This symbol represents entity types.
- **Ellipse:** These symbols represent attributes.
- **Dashed Ellipse:** These symbols represent derived attributes.
- **Diamond:** This symbol represents relationship types.
- **Lines:** It links attributes to entity types and entity types with other relationship types.
- **Primary key:** attributes are underlined
- **Double Ellipses:** Represent multi-valued attributes
- **Double Diamonds:** Represent identifying relationship.



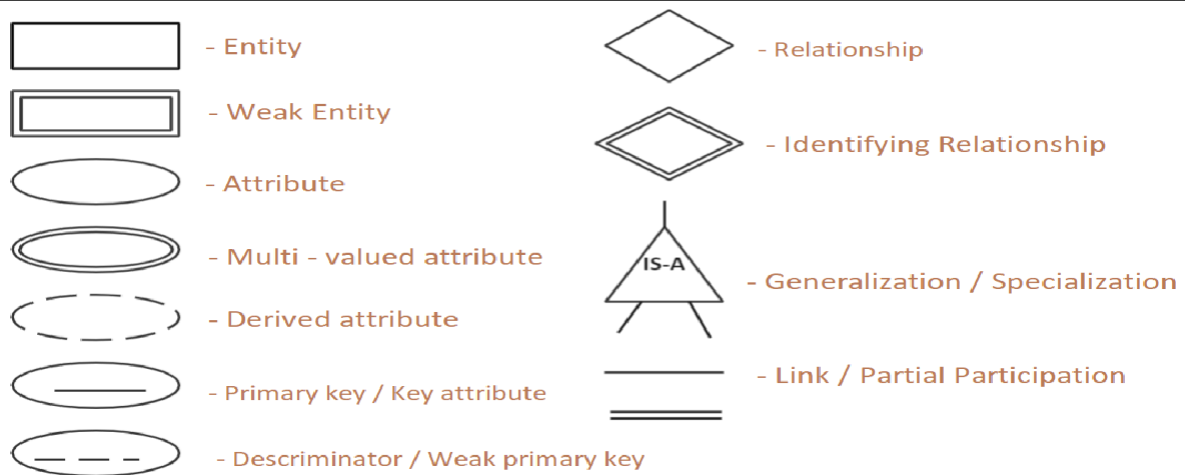


Fig. Symbols used in E – R Model

**Examples on Conceptual Design:**

1. In a university, a student enrolls in Courses. A student must be assigned to at least one or more Courses. Each course is taught by a single Professor. To maintain instruction quality, a professor can deliver only one course. Construct an E-R diagram for above problem.

**Steps to Create an ER Diagram**

1. Identify the Entities
2. Find relationships
3. Identify the cardinalities
4. Identify attributes including key attributes for every Entity
5. Draw complete E-R diagram with all attributes including Primary Key

**Step 1: Entity Identification.**

1. Student.
2. Course.
3. Professor.

**Step 2: Find Relationships.**

1. Student **enrolls** a course.
2. Course **taught** by professor

**Step 3: Identify the cardinalities.**

1. A student may enroll for more than one course (**one -to – many**).
2. Each course is taught by one course (**one – to - one**)

**Step 4: Identify attributes including key attributes for every Entity.**

| Entity    | Attributes                     |
|-----------|--------------------------------|
| Student   | <u>roll_no</u> , name, address |
| Course    | <u>c_id</u> , cname            |
| Professor | <u>emp_id</u> , ename, dept    |

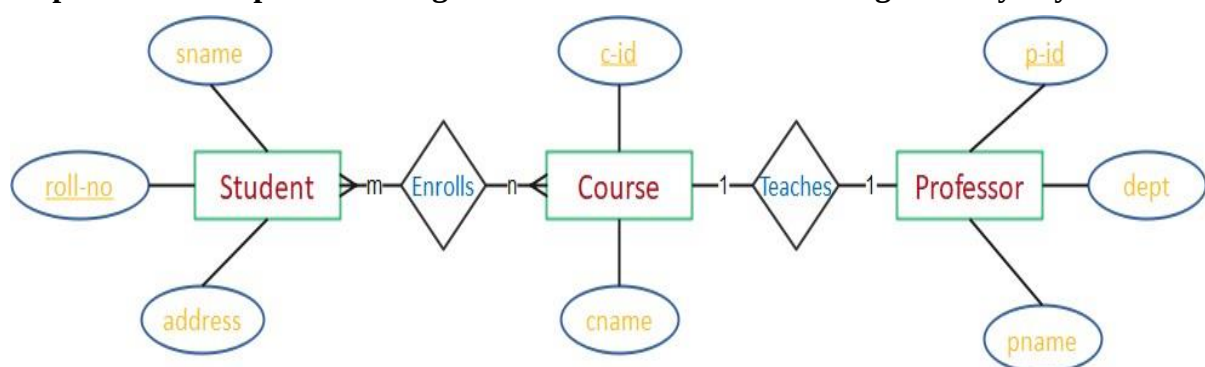
**Step 5: Draw complete E-R diagram with all attributes including Primary key.**

Fig. E-R Diagram

**2. Construct an E-R Model for a college database:** A college contains many departments. Each department can offer any number of courses. Many faculties can work in a department. A faculty can work only in one department. For each department there is a Head. A faculty can be head of only one department. Each faculty can take any number of courses. A course can be taken by only one faculty. A student can enroll for any number of courses. Each course can have any number of students.

**Step 1: Identify the Entities.** Department. Student. Course. Faculty.

**Step 2: Find the relationships:**

- Student **enrolls** for course.
- Department **offers** courses.
- Department **has** multiple instructors
- Each department is **supervised by** one faculty (**Head**).
- One course is **taught by** only one instructor.

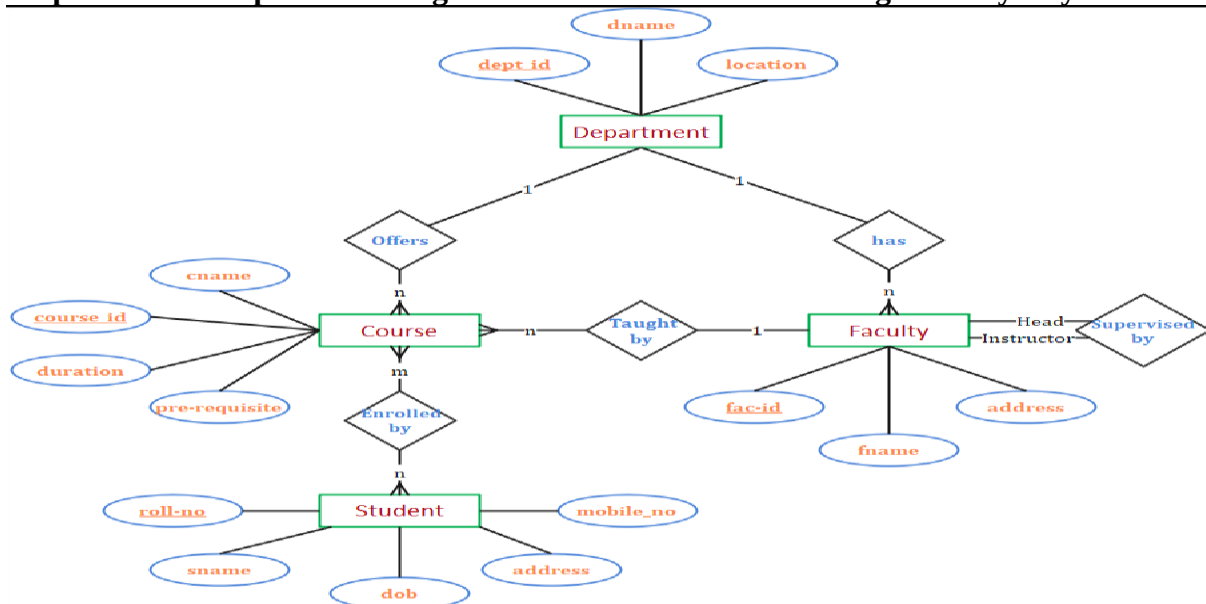
**Step 3: Identify the cardinalities**

- One course is enrolled by multiple students and one student enrolls for multiple courses, hence the cardinality between course and student is **Many to Many**.
- The department offers many courses and each course belongs to only one department, hence the cardinality between department and course is **one to many**.
- One department has multiple instructors and one instructor belongs to one and only one department, hence the cardinality between department and instructor is **one to many**.
- Each department there is a “Head of department” and one instructor is “Head of department”, hence the cardinality is **One to One**.
- One course is taught by only one instructor, but the instructor teaches many courses, hence the cardinality between course and instructor is **Many to One**.

**Step 4: Identify attributes including key attributes for every Entity.**

- For the Department entity, the attributes are dept\_id, dname, loc and here **dept\_id** is key attribute as it uniquely identifies every department.
- For the Course entity, the attributes are course-id, cname, duration, pre-requisite and here **course-id** is key attribute.
- For the Faculty entity, the attributes are fac\_id, fname, address, salary, and here **fac\_id** is key attribute.
- For the Student entity, the attributes are roll\_no, sname, DOB, address, mobile-no and here **roll\_no** is key attribute.

**Step 5: Draw complete E-R diagram with all attributes including Primary Key**



**3. Construct an E-R diagram for a Banking Management System.**

- ✓ To design the database for Banking Enterprise, first the requirements of Banking Enterprise are gathered and specified. From this requirement specification, the conceptual design of database is created. Here are the major characteristics of the banking enterprise;
  - The bank is organized into branches. Each branch is located in a particular city and is identified by a unique name. The bank monitors the assets of each branch.
  - Bank customers are identified by their *customer-id* values. The bank stores each customer's name, and the street and city where the customer lives. Customers may have accounts and can take out loans. A customer may be associated with a particular banker, who may act as a loan officer or personal banker for that customer.
  - Bank employees are identified by their *employee-id* values. The bank administration stores the name and telephone number of each employee, the names of the employee's dependents, and the *employee-id* number of the employee's manager. The bank also keeps track of the employee's start date and, thus, length of employment
  - The bank offers two types of accounts—savings and checking accounts. Accounts can be held by more than one customer, and a customer can have more than one account. Each account is assigned a unique account number. The bank maintains a record of each account's balance, and the most recent date on which the account was accessed by each customer holding the account. In addition, each savings account has an interest rate, and overdrafts are recorded for each checking account.
  - A loan originates at a particular branch and can be held by one or more customers. A loan is identified by a unique loan number. For each loan, the bank keeps track of the loan amount and the loan payments. Although a loan payment number does not uniquely identify a particular payment among those for all the bank's loans, a payment number does identify a particular payment for a specific loan. The date and amount are recorded for each payment
  - The bank would keep track of deposits and withdrawals from savings and checking accounts.

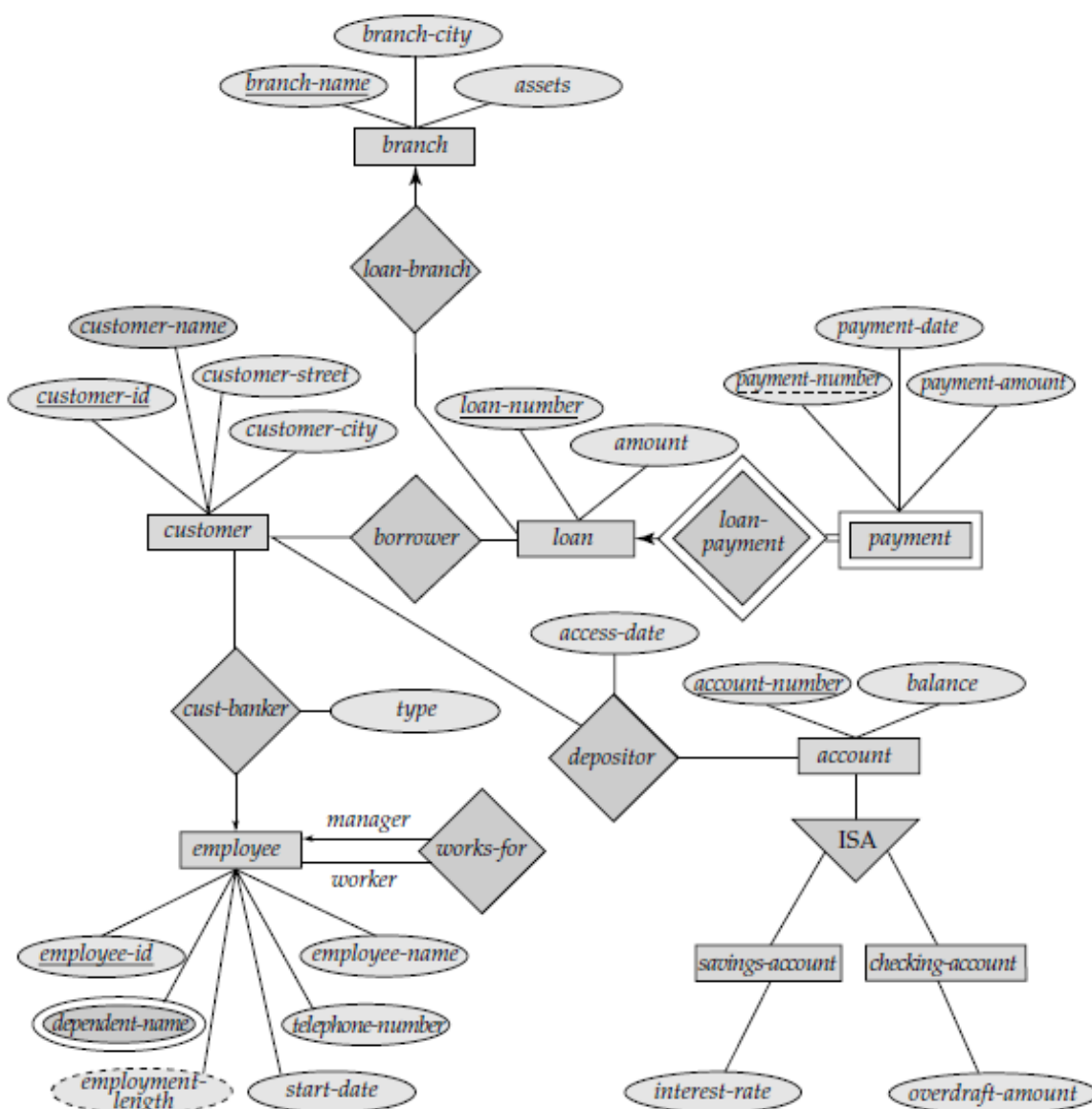
Keeping in mind the above requirements, the design of the database proceeds as follows:

**Step1: Identify the entity sets and their attributes**

- The entity sets that could be identified for Banking Enterprise are:
- *branch* entity set, with attributes *branch-name*, *branch-city*, and *assets*
- *customer* entity set, with attributes *customer-id*, *customer-name*, *customer-street*, and *customer-city*
- *Employee* entity set, with attributes *employee-id*, *employee-name*, *telephone-number*, *salary*, and *manager*. Additional descriptive features are the multi-valued attribute *dependent-name*, the base attribute *start-date*, and the derived attribute *employment-length*
- Two *account* entity sets—*savings-account* and *checking-account*—with the common attributes of *account-number* and *balance*; in addition, *savings-account* has the attribute *interest-rate* and *checking-account* has the attribute *overdraft-amount*
- The *loan* entity set, with the attributes *loan-number*, *amount*, and *originating-branch*
- The weak entity set *loan-payment*, with attributes *payment-number*, *payment-date*, and *payment-amount*.

**Step2: Identify the relationships and their cardinalities**

- *borrower*, a many-to-many relationship set between *customer* and *loan*
- *loan-branch* between *loan* and *branch*, a many-to-one relationship set that indicates in which branch a loan originated
- *loan-payment*, a one-to-many relationship from *loan* to *payment*, which documents that a payment is made on a loan
- *depositor*, with relationship attribute *access-date*, a many-to-many relationship set between *customer* and *account*, indicating that a customer owns an account
- *cust-banker*, with relationship attribute *type*, a many-to-one relationship set expressing that a customer can be advised by a bank employee, and that a bank employee can advise one or more customers
- *works-for*, a relationship set between *employee* entities with role indicators *manager* and *worker*; the mapping cardinalities express that an employee works for only one manager and that a manager supervises one or more employees

**Step3: Draw the E-R diagram with the identified entity sets and their relationships**

**(Extra Material for Understanding) – Not in Syllabus****Reduction to Relational Schemas**

- ✓ We can represent a database that conforms to an E-R database schema by a collection of tables.
- ✓ For each entity set and for each relationship set in the database, there is a unique table to which we assign the name of the corresponding entity set or relationship set.
- ✓ Each table has multiple columns, each of which has a unique name.
- ✓ Both the E-R model and the relational-database model are abstract, logical representations of real-world enterprises.
- ✓ Converting a database representation from an E-R diagram to a table format is the way we arrive at a relational-database design from an E-R diagram.

**Converting ER Diagrams to Tables-**

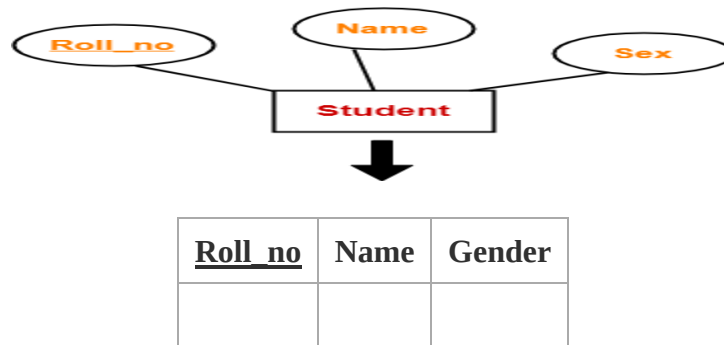
ER diagram is converted into the tables in relational model.

- ✓ This is because relational models can be easily implemented by RDBMS like MySQL, Oracle etc.

**Rule-01: For Strong Entity Set With Only Simple Attributes-**

A strong entity set with only simple attributes will require only one table in relational model.

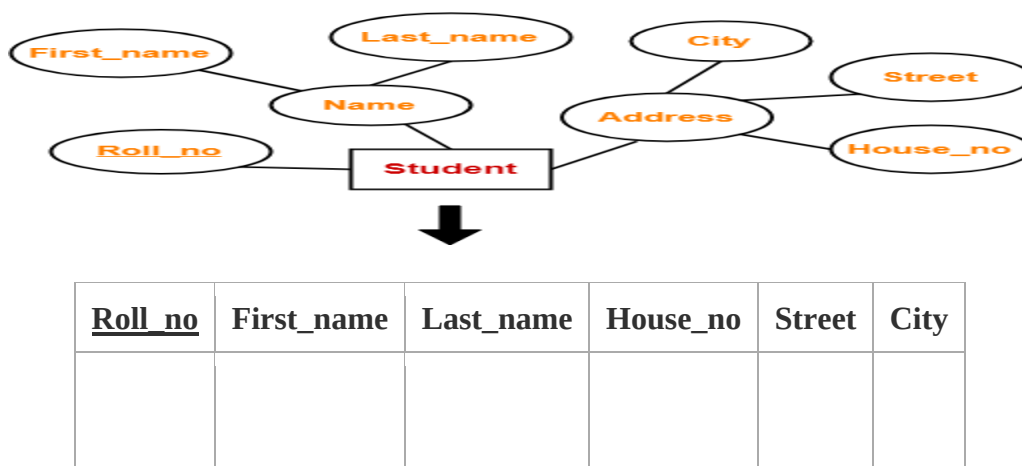
- ✓ Attributes of the table will be the attributes of the entity set.
- ✓ The primary key of the table will be the key attribute of the entity set.

**Example-**

**Schema: Student (Roll\_no, Name, Gender)**

**Rule-02: For Strong Entity Set With Composite Attributes-**

- A strong entity set with any number of composite attributes will require only one table in relational model.
- While conversion, simple attributes of the composite attributes are taken into account and not the composite attribute it.

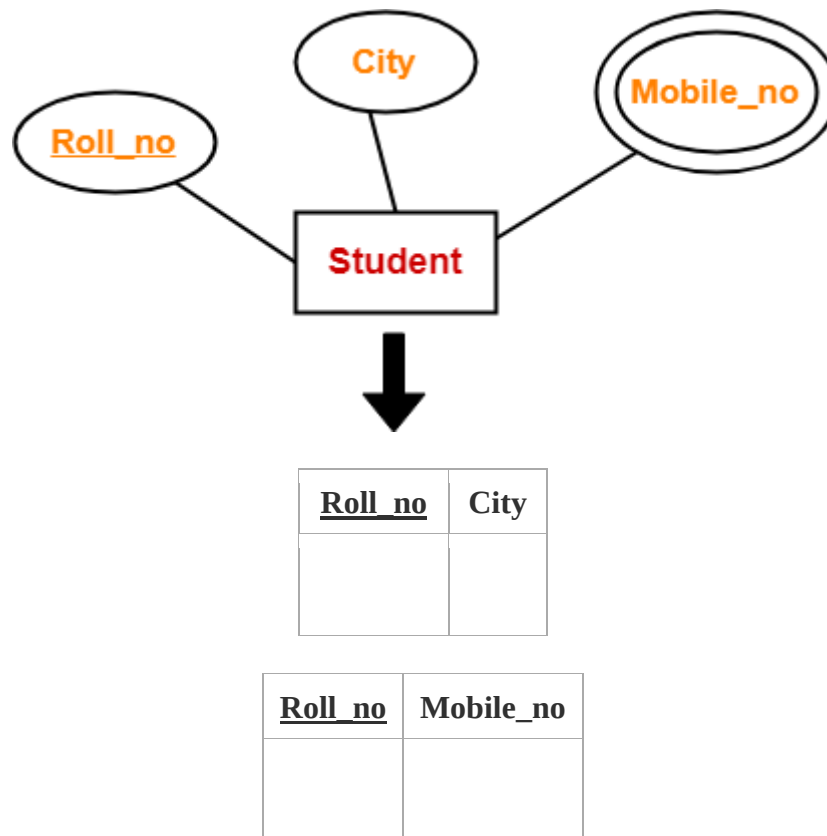
**Example-**

**Schema: Student (Roll\_no, First\_name, Last\_name, House\_no, Street, City)**

**Rule-03: For Strong Entity Set With Multi Valued Attributes-**

A strong entity set with any number of multi valued attributes will require two tables in relational model.

- ✓ One table will contain all the simple attributes with the primary key.
- ✓ Other table will contain the primary key and all the multi valued attributes.

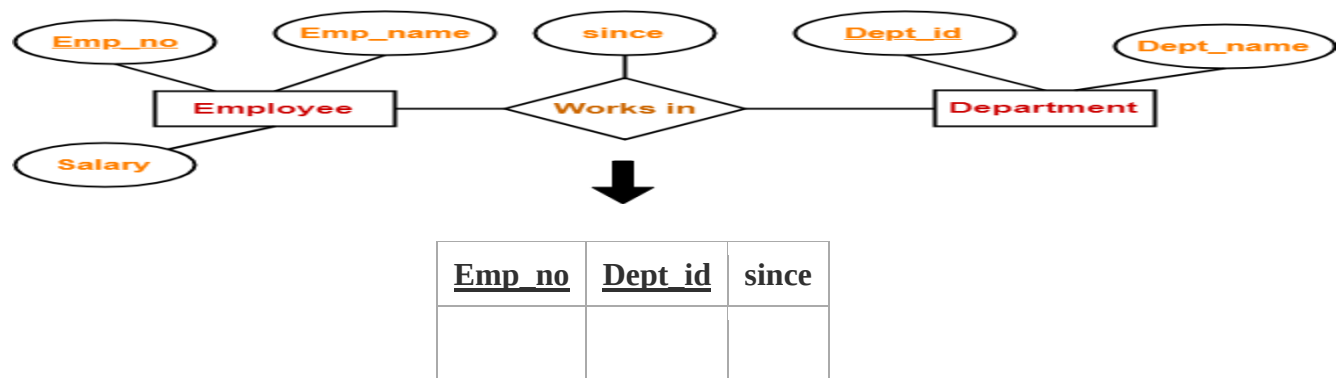
**Example-****Rule-04: Translating Relationship Set into a Table-**

A relationship set will require one table in the relational model.

Attributes of the table are-

- ✓ Primary key attributes of the participating entity sets
- ✓ Its own descriptive attributes if any.

Set of non-descriptive attributes will be the primary key.

**Example-**

Schema: Works in (Emp\_no, Dept\_id, since)

**NOTE-**

If we consider the overall ER diagram, three tables will be required in relational model-

- ✓ One table for the entity set “Employee”
- ✓ One table for the entity set “Department”
- ✓ One table for the relationship set “Works in”

**Rule-05: For Binary Relationships with Cardinality Ratios-**

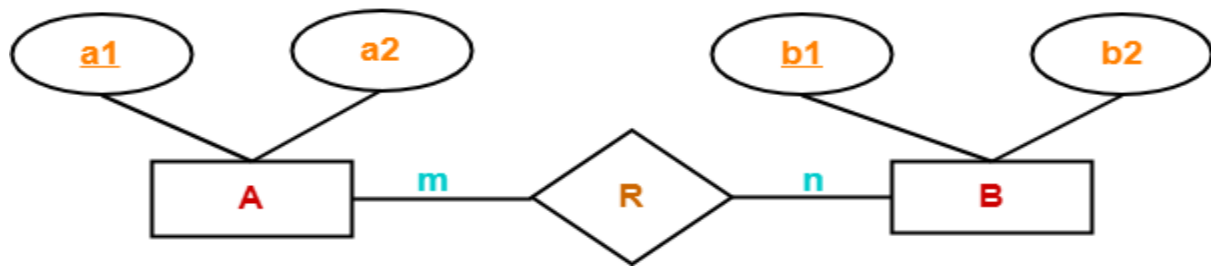
The following four cases are possible-

**Case-01:** Binary relationship with cardinality ratio m: n

**Case-02:** Binary relationship with cardinality ratio 1: n

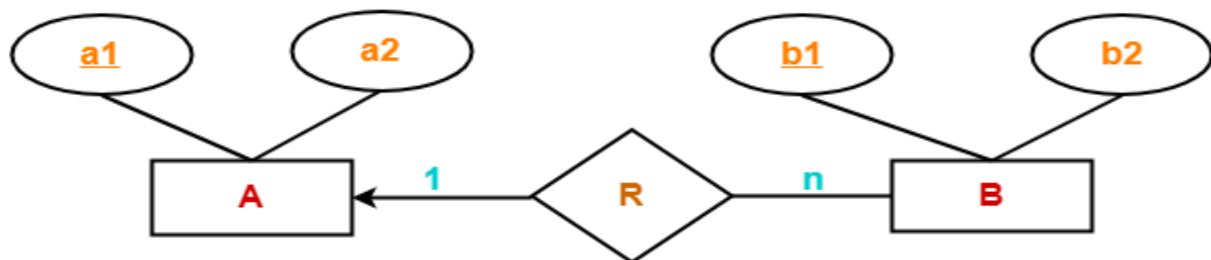
**Case-03:** Binary relationship with cardinality ratio m: 1

**Case-04:** Binary relationship with cardinality ratio 1: 1

**Case-01: For Binary Relationship with Cardinality Ratio m: n**

Here, three tables will be required-

1. A ( a1 , a2 )
2. R ( a1 , b1 )
3. B ( b1 , b2 )

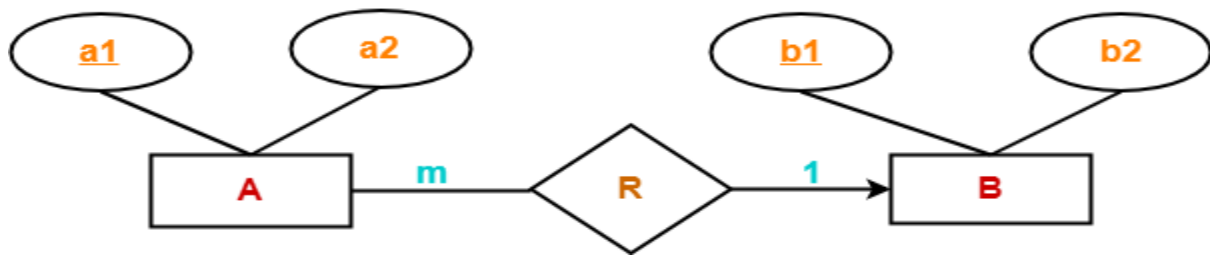
**Case-02: For Binary Relationship with Cardinality Ratio 1: n**

Here, two tables will be required-

1. A ( a1 , a2 )
2. BR ( a1 , b1 , b2 )

**NOTE-** Here, combined table will be drawn for the entity set B and relationship set R.

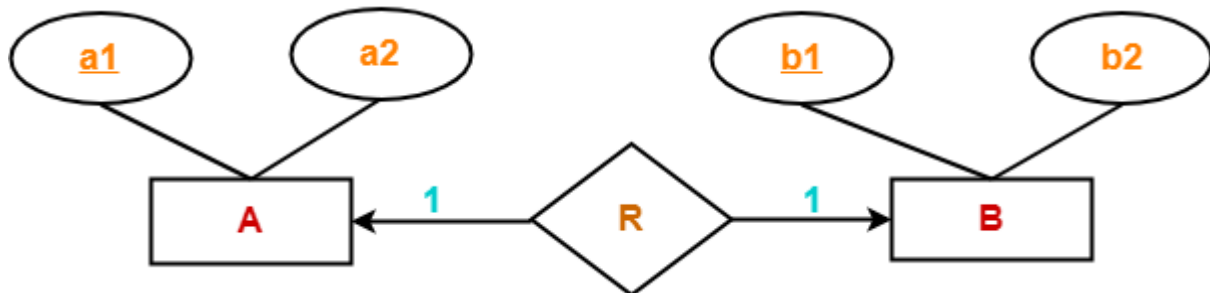


**Case-03: For Binary Relationship with Cardinality Ratio m: 1**

Here, two tables will be required-

1. AR ( a1 , a2 , b1 )
2. B ( b1 , b2 )

**NOTE-** Here, combined table will be drawn for the entity set A and relationship set R.

**Case-04: For Binary Relationship with Cardinality Ratio 1:1**

Here, two tables will be required. Either combines 'R' with 'A' or 'B'

**Way-01:**

1. A ( a1 , a2 )
2. BR ( a1, b1, b2)

**Way-02:**

1. AR ( a1 , a2 , b1 )
2. B ( b1 , b2 )

**Thumb Rules to Remember**

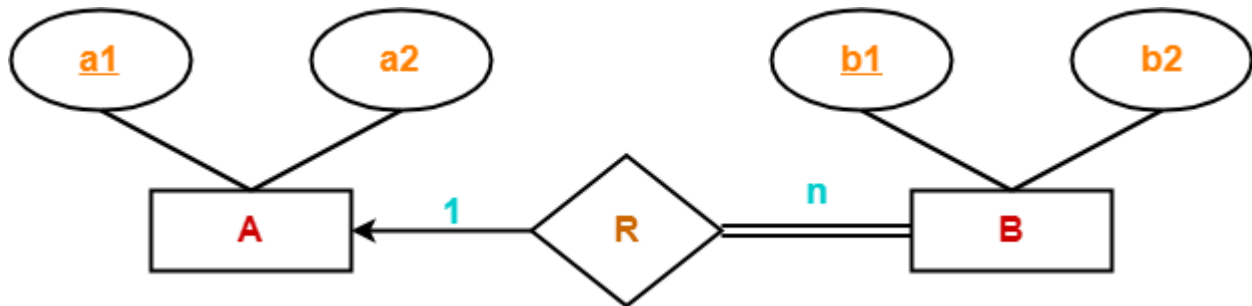
While determining the minimum number of tables required for binary relationships with given cardinality ratios, following thumb rules must be kept in mind-

- ✓ For binary relationship with cardinality ratio m: n, separate and individual tables will be drawn for each entity set and relationship.
- ✓ For binary relationship with cardinality ratio either m: 1 or 1: n, always remember "many side will consume the relationship" i.e. a combined table will be drawn for many side entity set and relationship set.
- ✓ For binary relationship with cardinality ratio 1: 1, two tables will be required. You can combine the relationship set with any one of the entity sets.

**Rule-06: For Binary Relationship with Both Cardinality Constraints and Participation Constraints-**

- ✓ Cardinality constraints will be implemented as discussed in Rule-05.
- ✓ Because of the total participation constraint, foreign key acquires **NOT NULL** constraint i.e. now foreign key cannot be null.

**Case-01: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from One Side-**



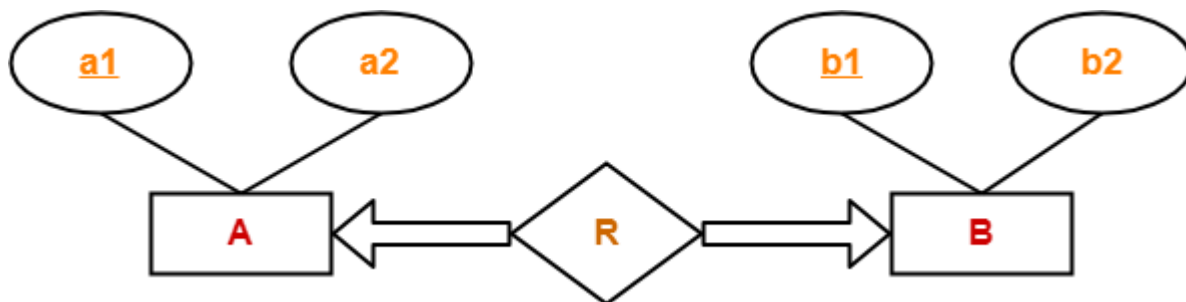
Because cardinality ratio = 1: n, so we will combine the entity set B and relationship set R.  
Then, two tables will be required-

1. A ( a1 , a2 )
2. BR ( a1 , b1 , b2 )

Because of total participation, foreign key a1 has acquired NOT NULL constraint, so it can't be null now.

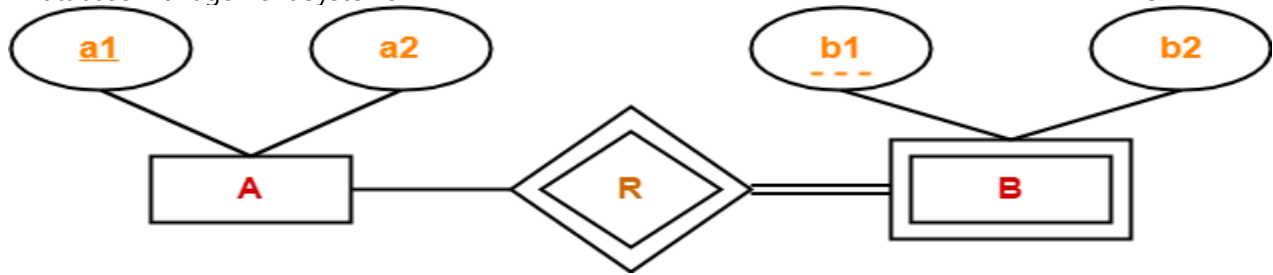
**Case-02: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from Both Sides-**

If there is a key constraint from both the sides of an entity set with total participation, then that binary relationship is represented using only single table.



**Rule-07: For Binary Relationship with Weak Entity Set-**

Weak entity set always appears in association with identifying relationship with total participation constraint.



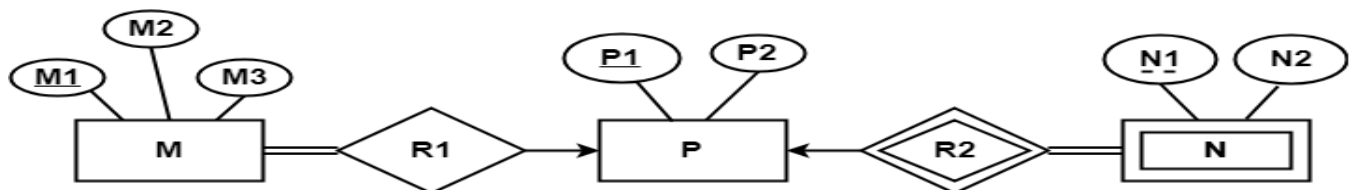
Here, two tables will be required-

1. A ( a1 , a2 )
2. BR ( a1 , b1 , b2 )

### PRACTICE PROBLEMS BASED ON CONVERTING ER DIAGRAM TO TABLES-

#### Problem-01:

Find the minimum number of tables required for the following ER diagram in relational model-



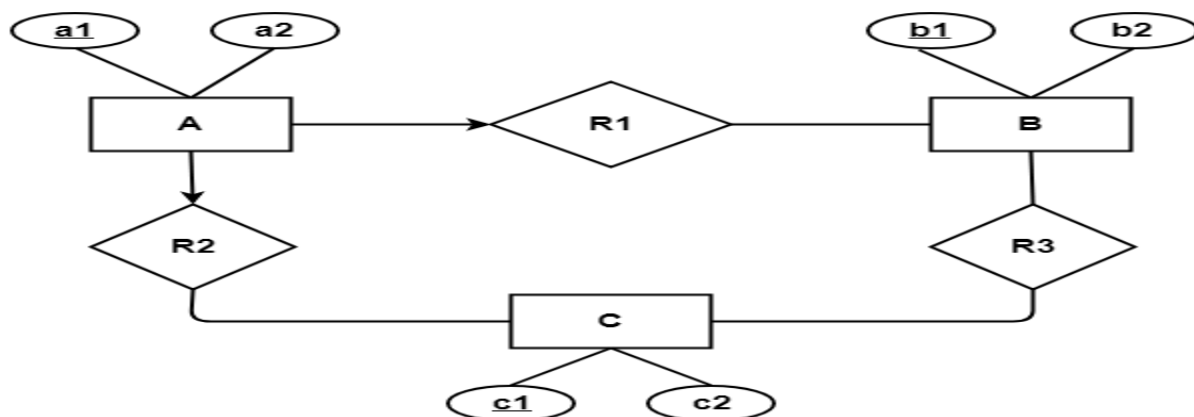
#### Solution-

Applying the rules, minimum 3 tables will be required-

- MR1 ( M1 , M2 , M3 , P1 )
- P ( P1 , P2 )
- NR2 ( P1 , N1 , N2 )

#### Problem-02:

Find the minimum number of tables required to represent the given ER diagram in relational model-



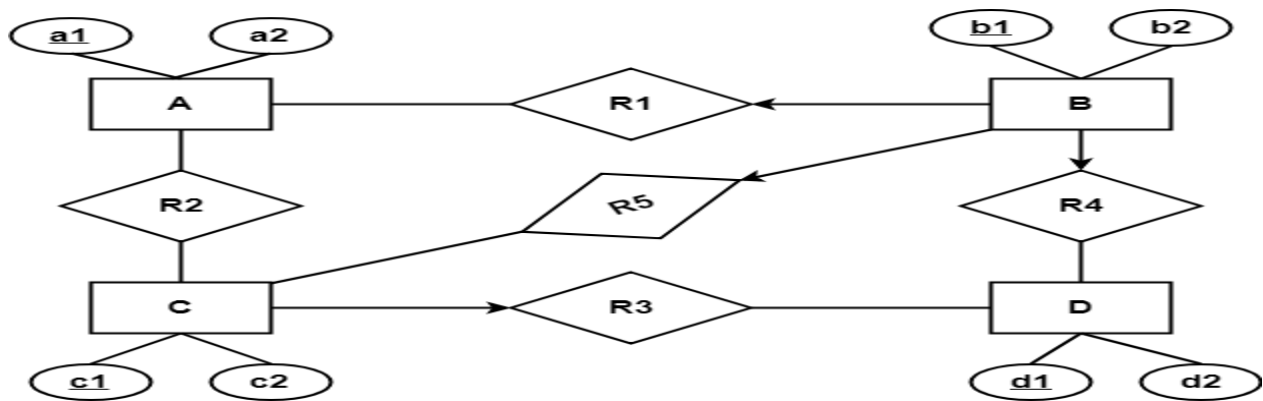
#### Solution-

Applying the rules, minimum 4 tables will be required-

- AR1R2 ( a1 , a2 , b1 , c1 )
- B ( b1 , b2 )
- C ( c1 , c2 )
- R3 ( b1 , c1 )

**Problem-03:**

Find the minimum number of tables required to represent the given ER diagram in relational model-

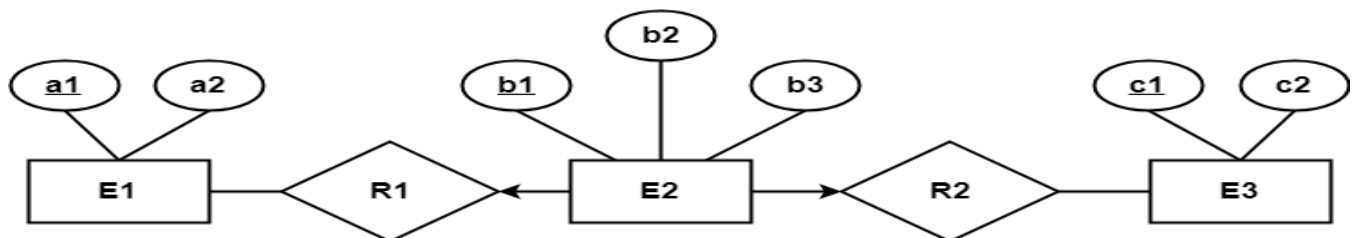
**Solution-**

Applying the rules, minimum 5 tables will be required-

- BR1R4R5 (b1 , b2 , a1 , c1 , d1)
- A (a1 , a2)
- R2 (a1 , c1)
- CR3 (c1 , c2 , d1)
- D (d1 , d2)

**Problem-04:**

Find the minimum number of tables required to represent the given ER diagram in relational model-

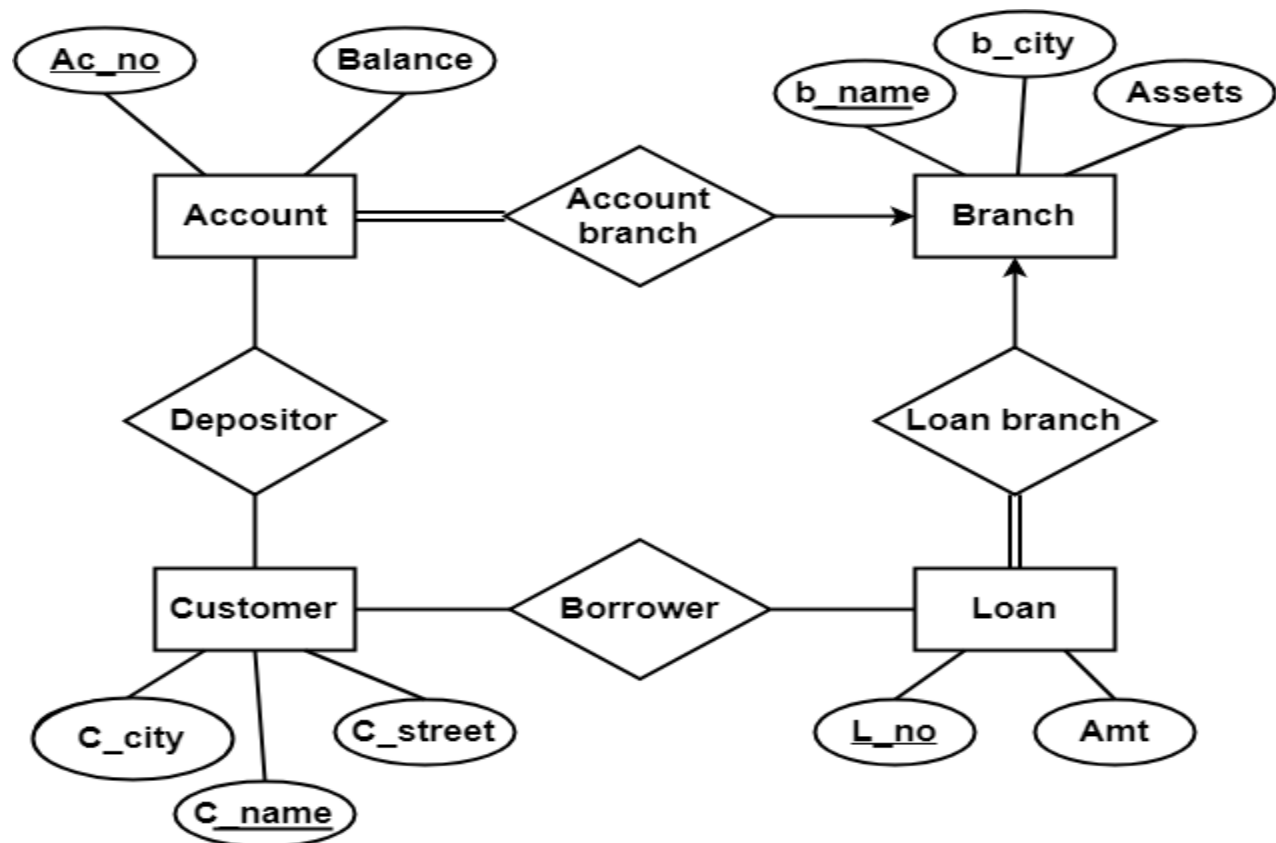
**Solution-**

Applying the rules, minimum 3 tables will be required-

- E1 (a1 , a2)
- E2R1R2 (b1 , b2 , a1 , c1 , b3)
- E3 (c1 , c2)

**Problem-05:**

Find the minimum number of tables required to represent the given ER diagram in relational model-

**Solution-**

Applying the rules that we have learnt, minimum 6 tables will be required-

- Account (Ac\_no , Balance , b\_name)
- Branch (b\_name , b\_city , Assets)
- Loan (L\_no , Amt , b\_name)
- Borrower (C\_name , L\_no)
- Customer (C\_name , C\_street , C\_city)
- Depositor (C\_name , Ac\_no)